

人流シミュレーションで学ぶ 避難誘導最適化

(筑波大学 全学計算機用)

産業技術総合研究所 人工知能研究センター
筑波大学大学院（連携大学院）
大西 正輝

実行のながれ

JAVA で書かれた CrowdWalk を Python で動かす

産総研で開発している
人流シミュレータ

- ① CrowdWalk の講義用パッケージをダウンロード・展開
- ② Python を動かすために Jupyter Notebook を起動
- ③ Jupyter から CrowdWalkを実行

環境要件(筑波大学全学計算機は標準で満たす)

- Anaconda3
- Jupyter Notebook
- 7-Zip(Windows標準の機能でも可)

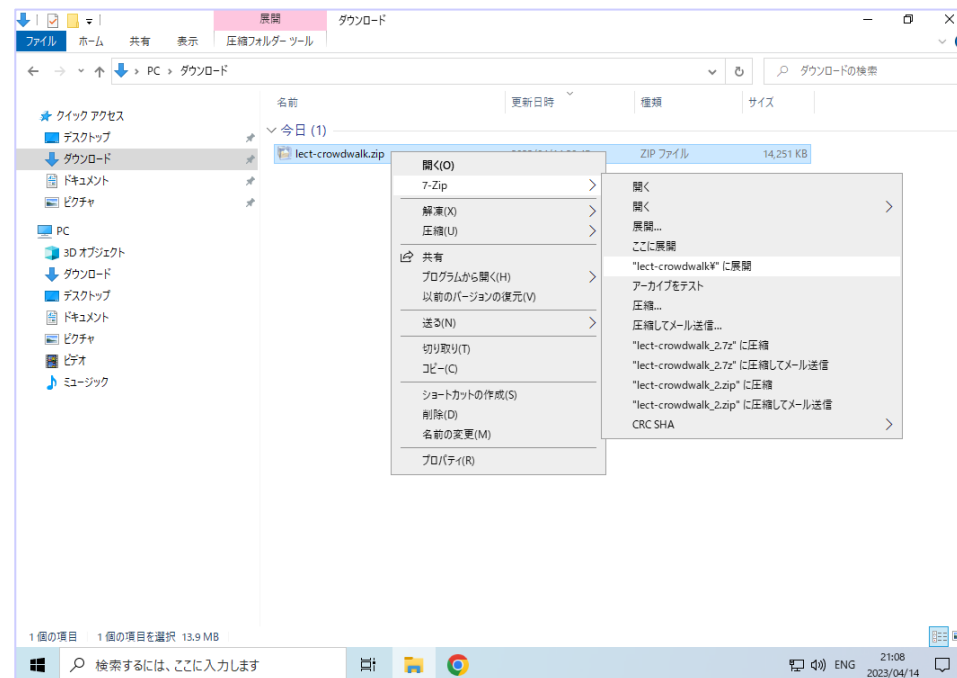
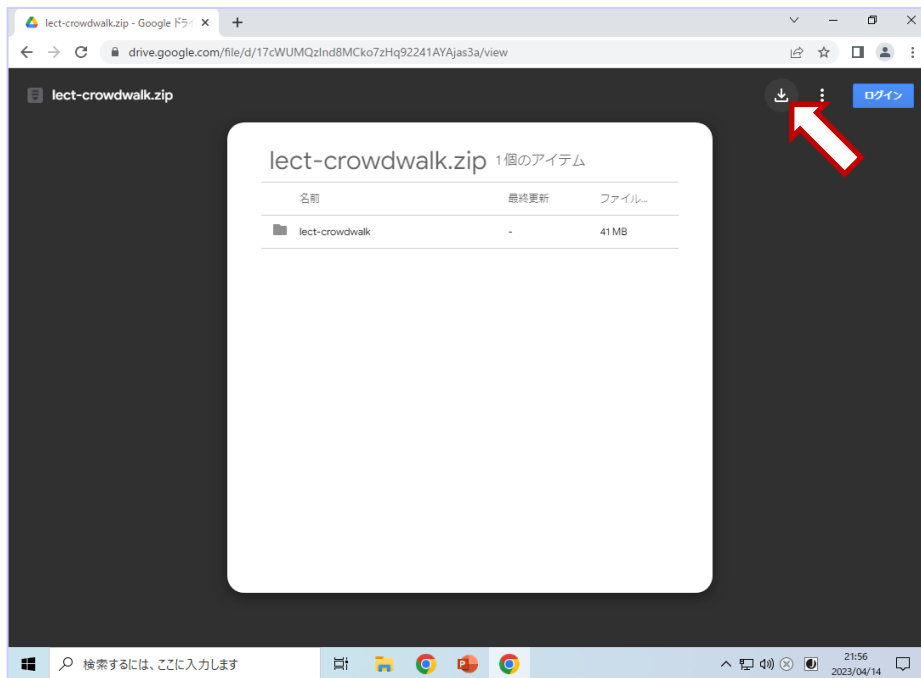
① 講義用パッケージをダウンロード・展開

下記のURLにアクセスしてダウンロード

<https://bit.ly/lect-cw>

7-Zipを使って”lect-crowdwalk¥”に展開

※ win で zip はダブルクリックしても解凍されません！
(右クリックで展開・解凍して下さい)

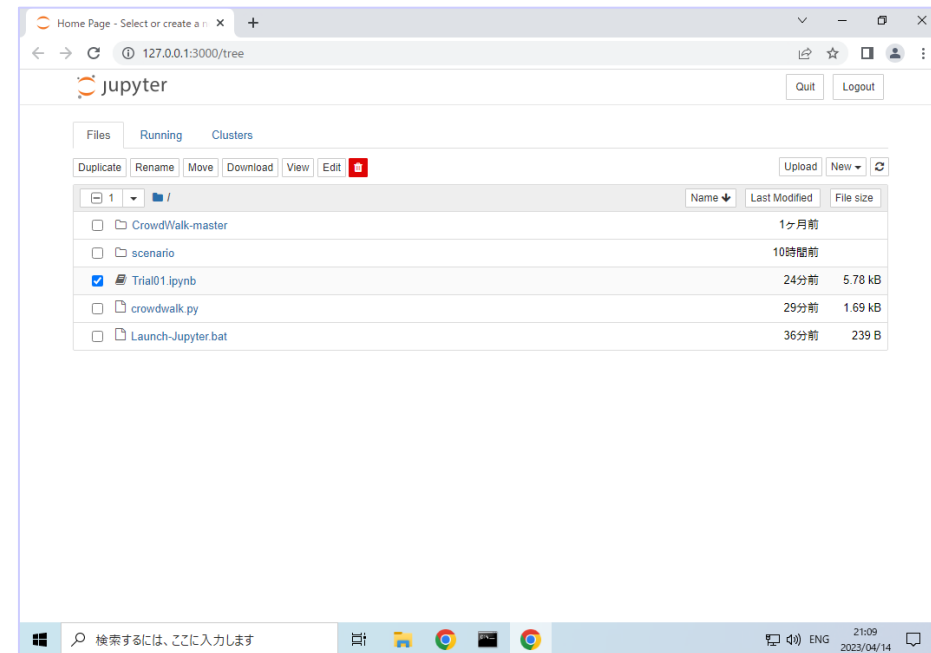
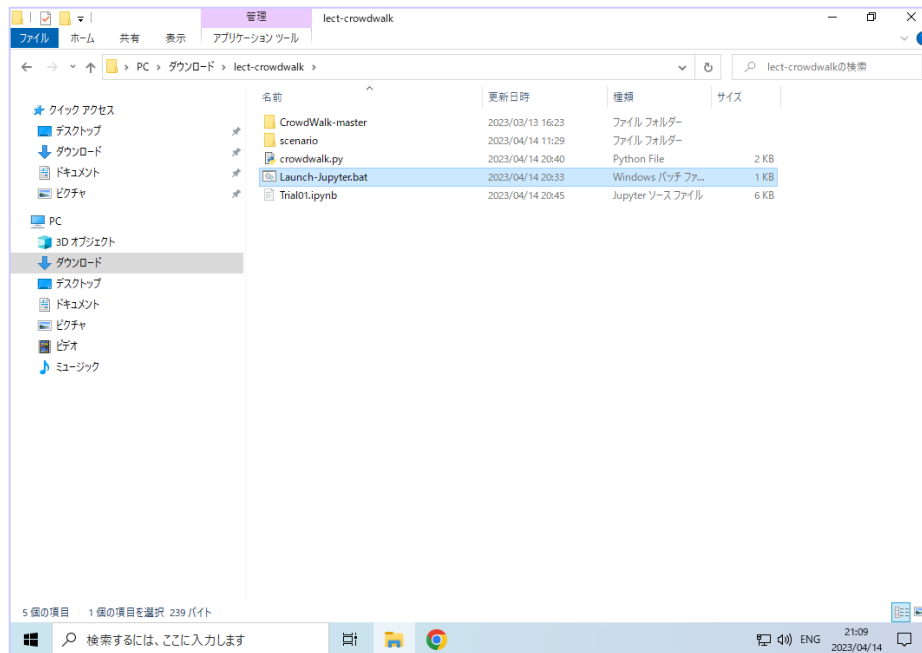


② Python を動かすために Jupyter Notebook を起動

「lect-crowdwalk」を開いて
「Launch-Jupyter.bat」を実行

- ※ 拡張子が見えない設定になっている人が多い
- ※ Trial01.ipynbを直接開かないこと

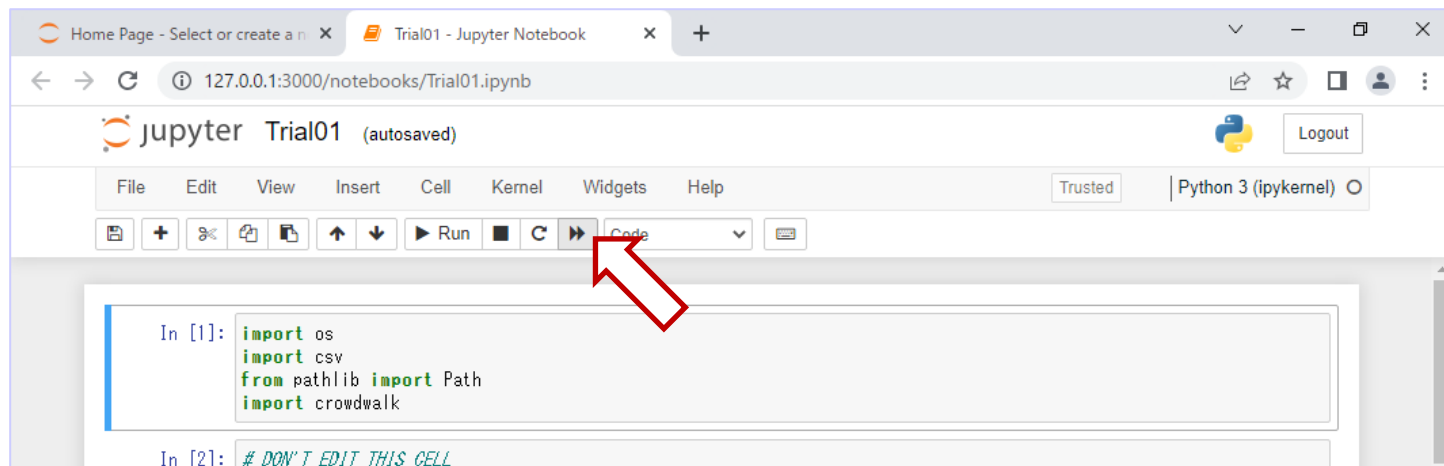
Jupyterで「Trial01.ipynb」を開く



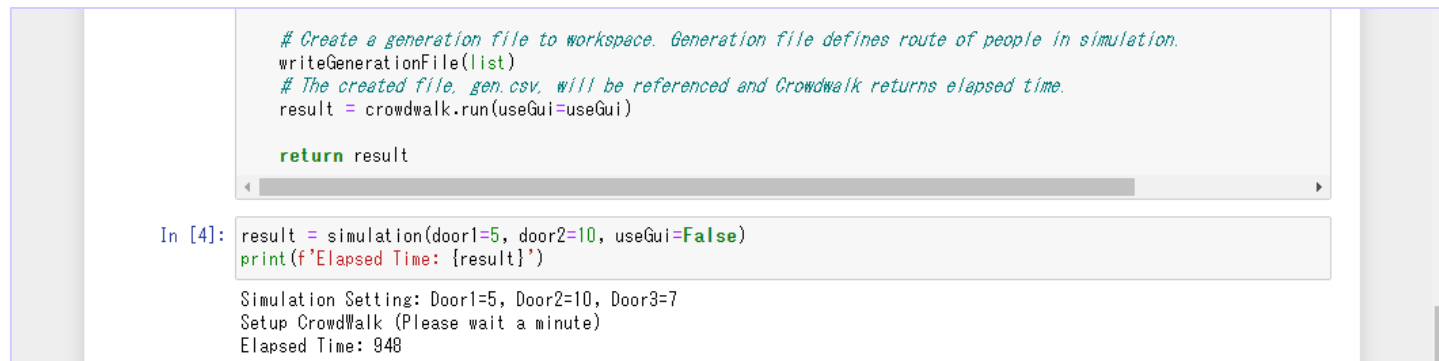
Mac で動かす場合は自己責任で
JDK17のインストール、Liberica JDK 17 x86 のインストール

③ Jupyter から CrowdWalkを実行

起動直後の実行はRestartボタンをクリック
(起動直後の実行はセットアップが必要に応じて実行されるため時間がかかる)



結果(Elapsed Time)が出力されたらOK



シミュレーションが終わると
避難完了時間が入る

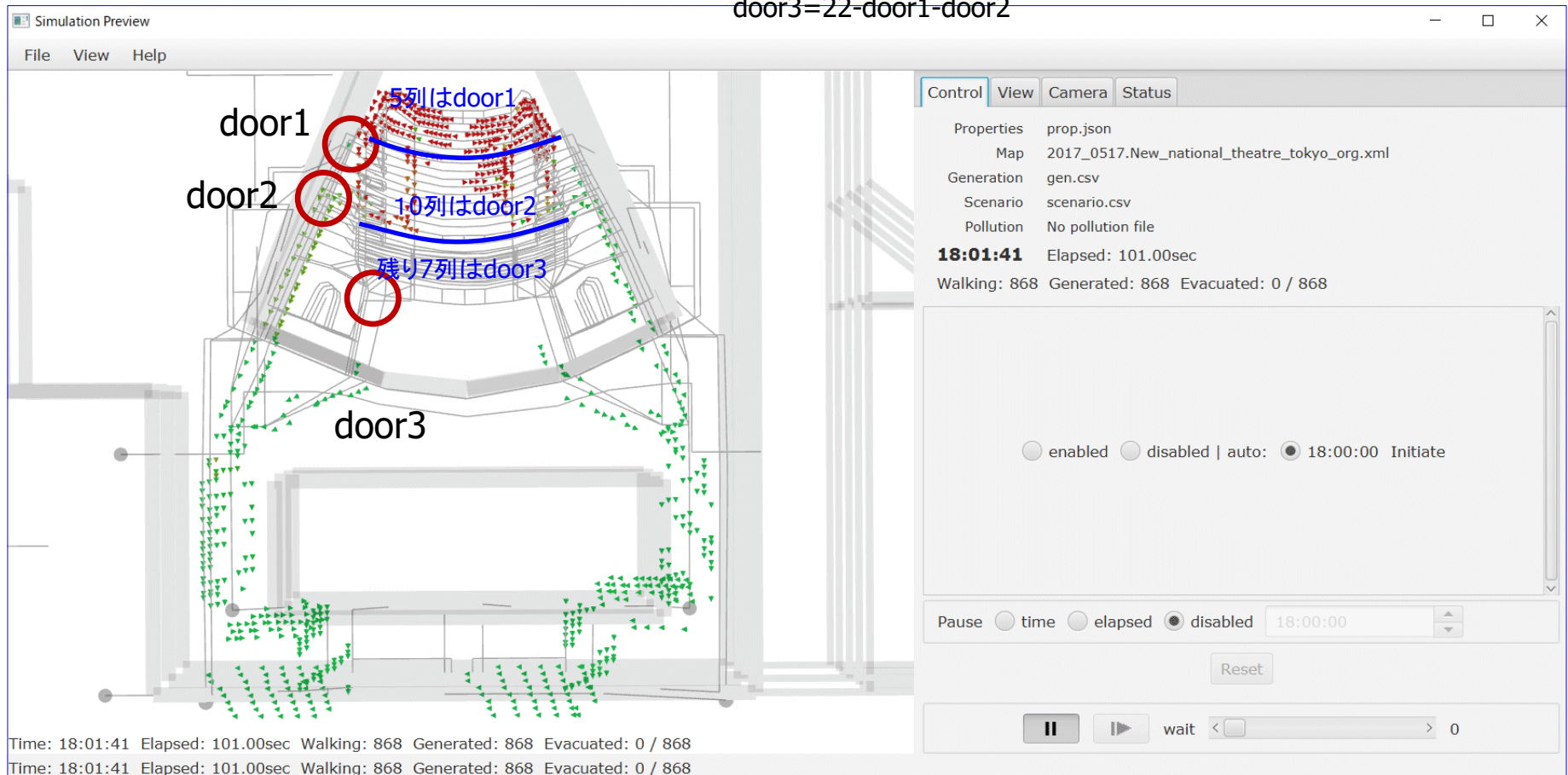
door1
から出る
列数

door2
から出る
列数

描画するなら True
しないなら False

```
In [4]: result = simulation(door1=5, door2=10, use_gui=True)
print(f'Elapsed Time: {result}')
```

残りは door3 から出る
 $\text{door3} = 22 - \text{door1} - \text{door2}$



最も速く避難できる誘導方法をシミュレーションで導出してください

【疑似コード】

```

mini_time に十分に大きな数字を入れる
door1 に 0~22 の数字をいれてループを回す{
    door2 に 0~22-door1 をいれてループを回す{
        シミュレーションを回す
        もし 避難時間が mini_time よりも小さい場合には{
            mini_time と mini_door1 と mini_door2 を記録
        }
    }
}

```

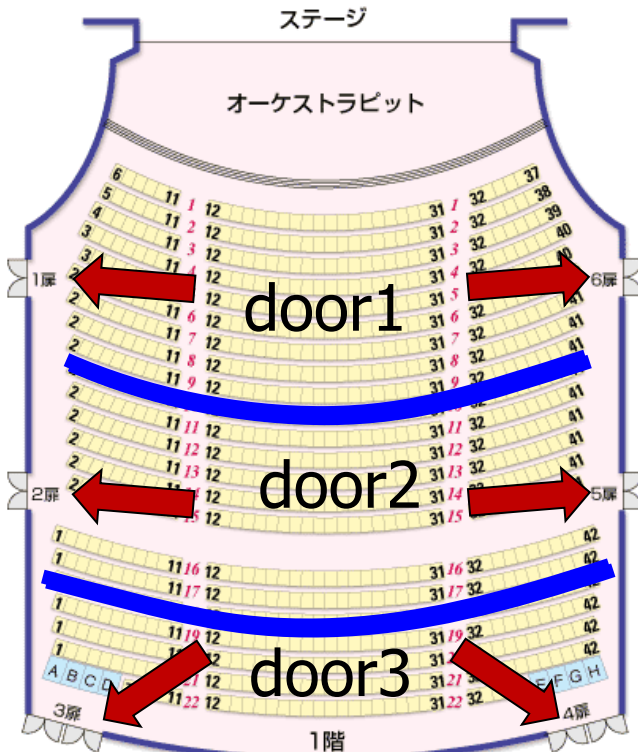
全てのループが終わったら

Mini_door1, mini_door2, mini_door3, mini_time を表示

```

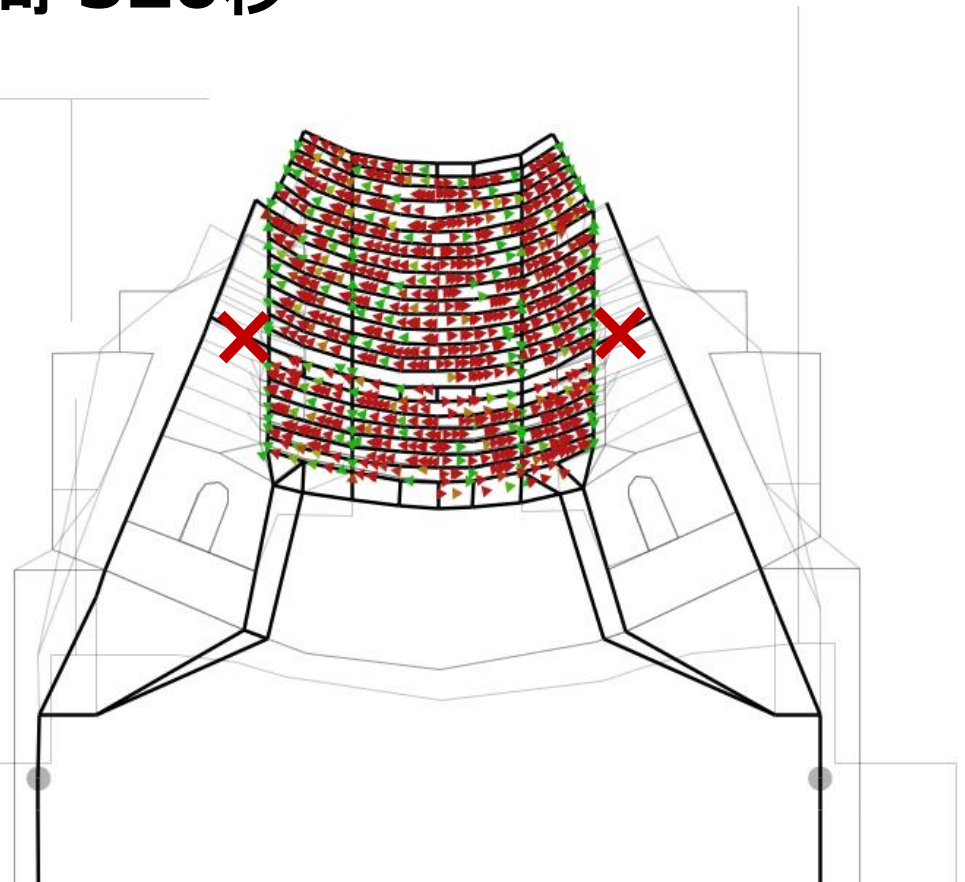
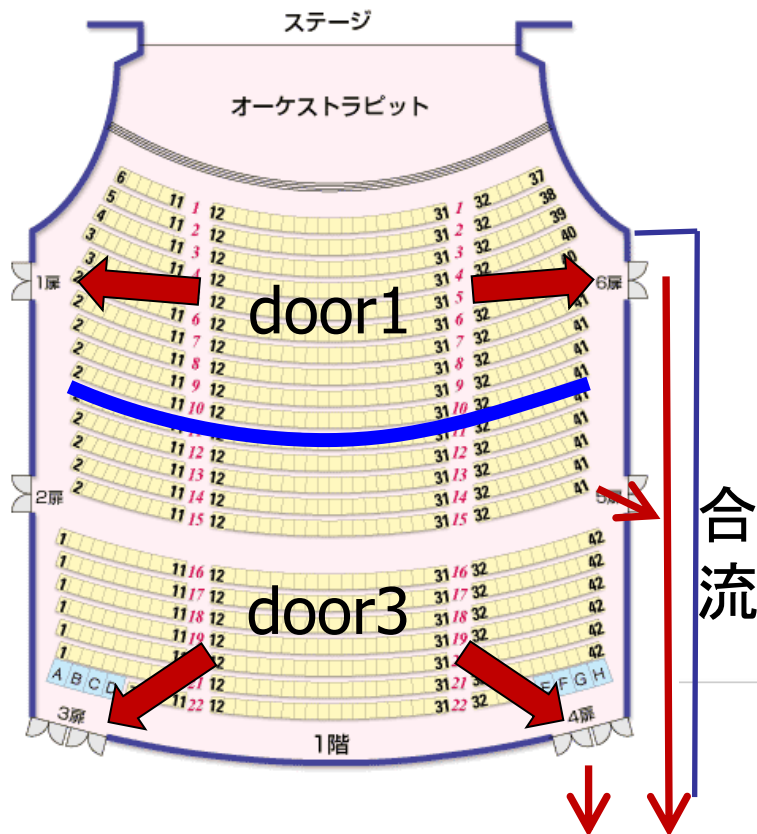
min_time = 1000000
for door1 in range(0,23,1):
    for door2 in range(0,23-door1,1):
        result = simulation(door1, door2, use_gui=False)
        if result < min_time:
            min_time = result
            min_door1 = door1
            min_door2 = door2
        print(f'^Elapsed Time: {result}')
print("最適誘導", min_door1, min_door2, 22-min_door1-min_door2)

```



最も速く避難できる誘導方法をシミュレーションで導出してください

**解答: door1 10列, door3 12列, door2 は使わない
場合に避難完了時間 326秒**



さらに複雑な制御を行うための理解

```
def simulation(door1, door2, use_gui=False):
    door3 = 22 - door1 - door2

    print(f'Simulation Setting: Door1={door1}, Door2={door2}, Door3={door3}')

    if door3 < 0:
        raise ValueError(f'Invalid inputs: 22 - door1({door1}) - door2({door2}) = door3({door3}).')

    list = []

    # Generate agents that are exit from door 1
    for row in range(1, (door1 + 1)):
        for column in range(1, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door1Left"]))
        for column in range(4, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door1Right"]))

    # Generate agents that are exit from door 2
    for row in range((door1 + 1), (door1 + door2 + 1)):
        for column in range(1, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door2Left"]))
        for column in range(4, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door2Right"]))

    # Generate agents that are exit from door 3
    for row in range((door1 + door2 + 1), (door1 + door2 + door3 + 1)):
        for column in range(1, (2 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3OuterLeft"]))
        for column in range(3, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3InnerLeft"]))
        for column in range(4, (4 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3InnerRight"]))
        for column in range(5, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3OuterRight"]))

    # Create a generation file to workspace. The generation file defines route of people in simul.
    write_generation_file(list)
    # The created file, gen.csv, will be referenced and Crowdwalk returns elapsed time.
    result = crowdwalk.run(use_gui=use_gui)

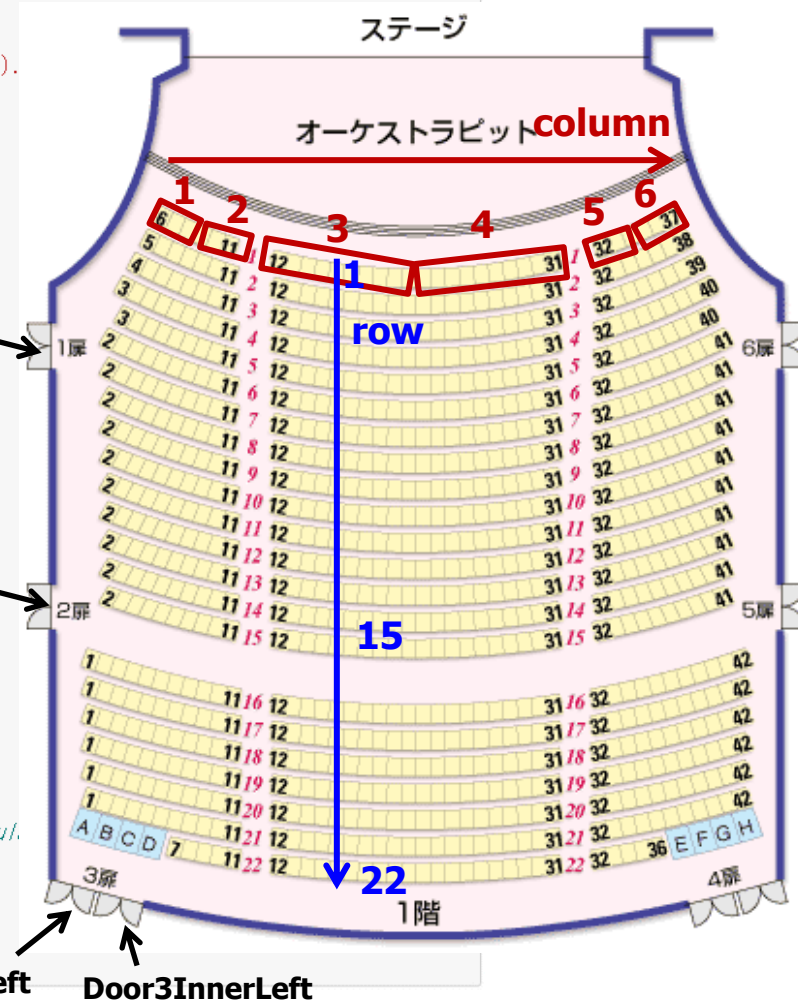
    return result
```

Door1Left

Door2Left

Door3OuterLeft

Door3InnerLeft



さらに複雑な制御を行うための理解

```
def simulation(door1, door2, use_gui=False):
    door3 = 22 - door1 - door2

    print(f'Simulation Setting: Door1={door1}, Door2={door2}, Door3={door3}')

    if door3 < 0:
        raise ValueError(f"Invalid inputs: 22 - door1({door1}) - door2({door2}) = door3({door3})")

    list = []

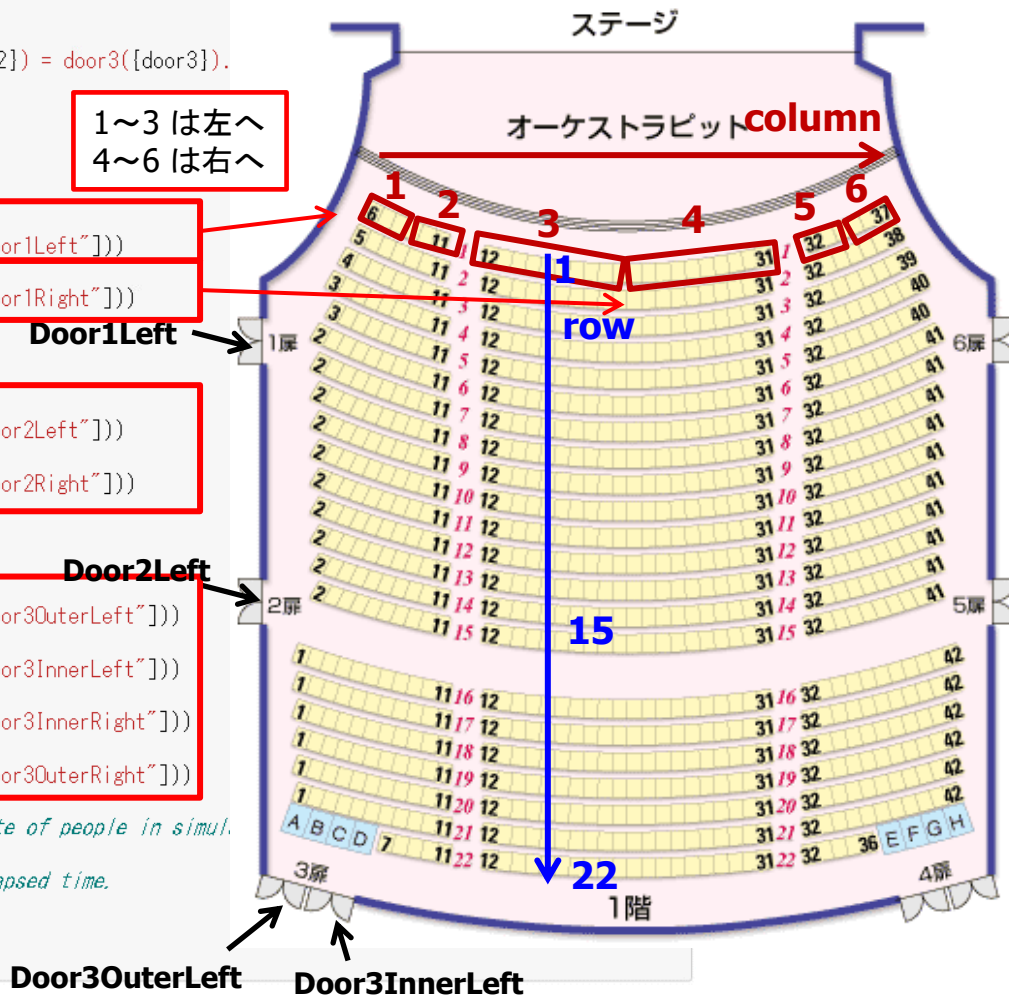
    # Generate agents that are exit from door 1
    for row in range(1, (door1 + 1)):
        for column in range(1, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door1Left"]))
        for column in range(4, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door1Right"]))

    # Generate agents that are exit from door 2
    for row in range((door1 + 1), (door1 + door2 + 1)):
        for column in range(1, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door2Left"]))
        for column in range(4, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door2Right"]))

    # Generate agents that are exit from door 3
    for row in range((door1 + door2 + 1), (door1 + door2 + door3 + 1)):
        for column in range(1, (2 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3OuterLeft"]))
        for column in range(3, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3InnerLeft"]))
        for column in range(4, (4 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3InnerRight"]))
        for column in range(5, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3OuterRight"]))

    # Create a generation file to workspace. The generation file defines route of people in simul.
    write_generation_file(list)
    # The created file, gen.csv, will be referenced and Crowdwalk returns elapsed time.
    result = crowdwalk.run(use_gui=use_gui)

    return result
```



さらに複雑な制御を行うための理解

```
def simulation(door1, door2, use_gui=False):
    door3 = 22 - door1 - door2

    print(f'Simulation Setting: Door1={door1}, Door2={door2}, Door3={door3}')

    if door3 < 0:
        raise ValueError(f'Invalid inputs: 22 - door1({door1}) - door2({door2}) = door3({door3}).')

    list = []

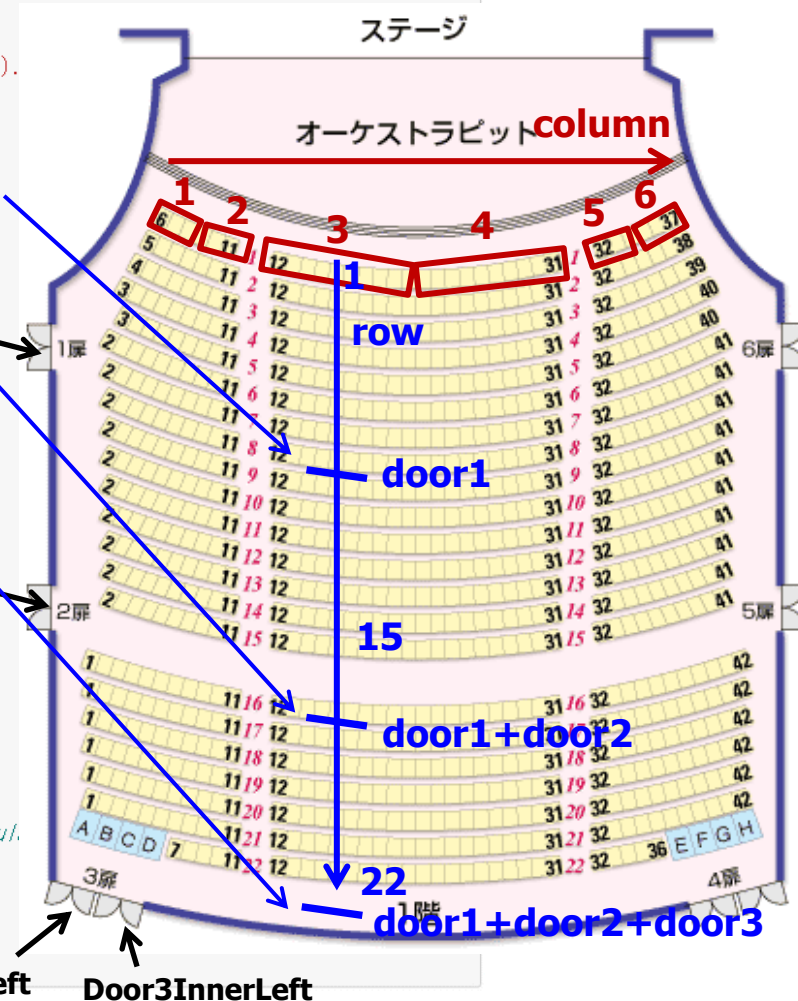
    # Generate agents that are exit from door 1
    for row in range(1, (door1 + 1)):
        for column in range(1, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door1Left"]))
        for column in range(4, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door1Right"]))

    # Generate agents that are exit from door 2
    for row in range((door1 + 1), (door1 + door2 + 1)):
        for column in range(1, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door2Left"]))
        for column in range(4, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door2Right"]))

    # Generate agents that are exit from door 3
    for row in range((door1 + door2 + 1), (door1 + door2 + door3 + 1)):
        for column in range(1, (2 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3OuterLeft"]))
        for column in range(3, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3InnerLeft"]))
        for column in range(4, (4 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3InnerRight"]))
        for column in range(5, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3OuterRight"]))

    # Create a generation file to workspace. The generation file defines route of people in simul.
    write_generation_file(list)
    # The created file, gen.csv, will be referenced and Crowdwalk returns elapsed time.
    result = crowdwalk.run(use_gui=use_gui)

    return result
```



さらに複雑な制御を行うための理解

```
def simulation(door1, door2, use_gui=False):
    door3 = 22 - door1 - door2

    print(f'Simulation Setting: Door1={door1}, Door2={door2}, Door3={door3}')

    if door3 < 0:
        raise ValueError(f'Invalid inputs: 22 - door1({door1}) - door2({door2}) = door3({door3}).')

    list = []

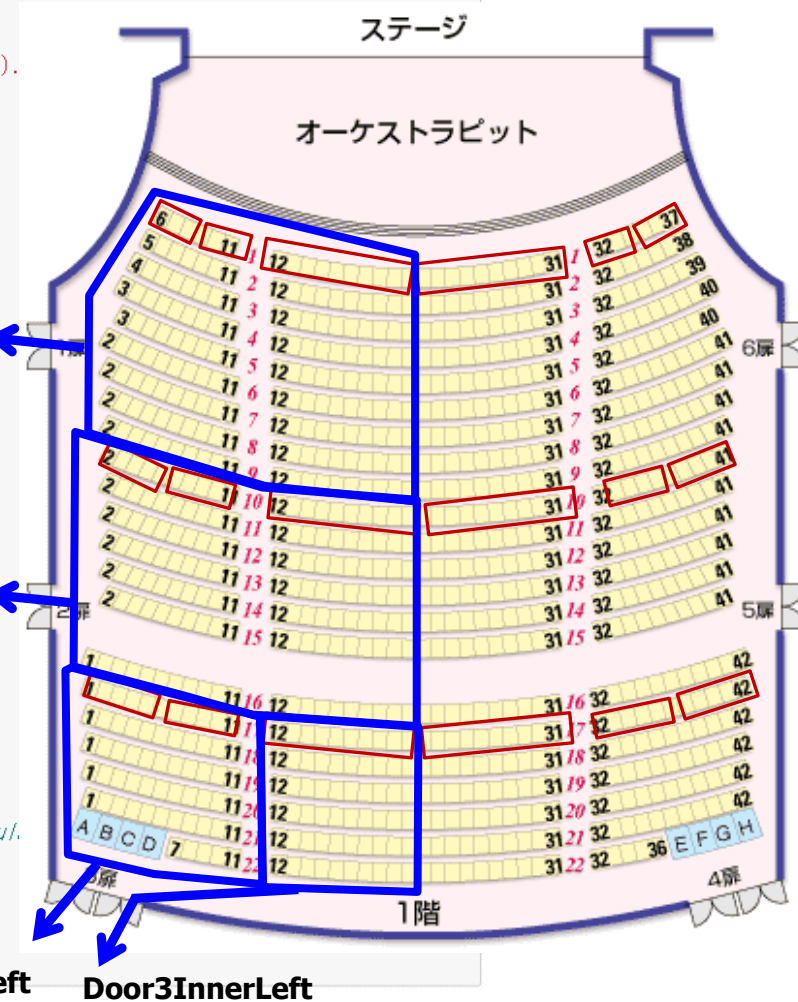
    # Generate agents that are exit from door 1
    for row in range(1, (door1 + 1)):
        for column in range(1, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door1Left"]))
        for column in range(4, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door1Right"]))

    # Generate agents that are exit from door 2
    for row in range((door1 + 1), (door1 + door2 + 1)):
        for column in range(1, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door2Left"]))
        for column in range(4, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door2Right"]))

    # Generate agents that are exit from door 3
    for row in range((door1 + door2 + 1), (door1 + door2 + door3 + 1)):
        for column in range(1, (2 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3OuterLeft"]))
        for column in range(3, (3 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3InnerLeft"]))
        for column in range(4, (4 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3InnerRight"]))
        for column in range(5, (6 + 1)):
            list.append(create_agent_rule(column, row, planned_route_list["Door3OuterRight"]))

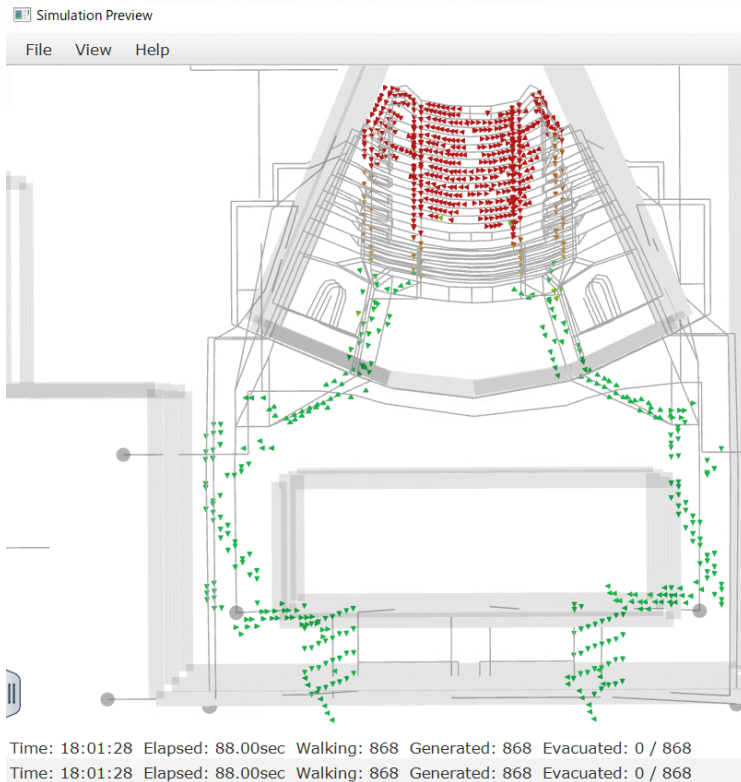
    # Create a generation file to workspace. The generation file defines route of people in simul.
    write_generation_file(list)
    # The created file, gen.csv, will be referenced and Crowdwalk returns elapsed time.
    result = crowdwalk.run(use_gui=use_gui)

    return result
```



避難状況の画像作成

lect-crowdwark¥work¥log¥cap に画像ができる



Simulation Preview

File View Help

Time: 18:01:28 Elapsed: 88.00sec Walking: 868 Generated: 868 Evacuated: 0 / 868
 Time: 18:01:28 Elapsed: 88.00sec Walking: 868 Generated: 868 Evacuated: 0 / 868

Control View Camera Status

Vertical scale < 1.0

Agent size < 0.8

☒ Show nodes

☒ Show links

☐ Show areas ☐ Show outline of areas

☒ Show agents

☐ Show polygons ☐ Show the sea

☐ Show background map

☒ Record simulation screen (800 x 600)

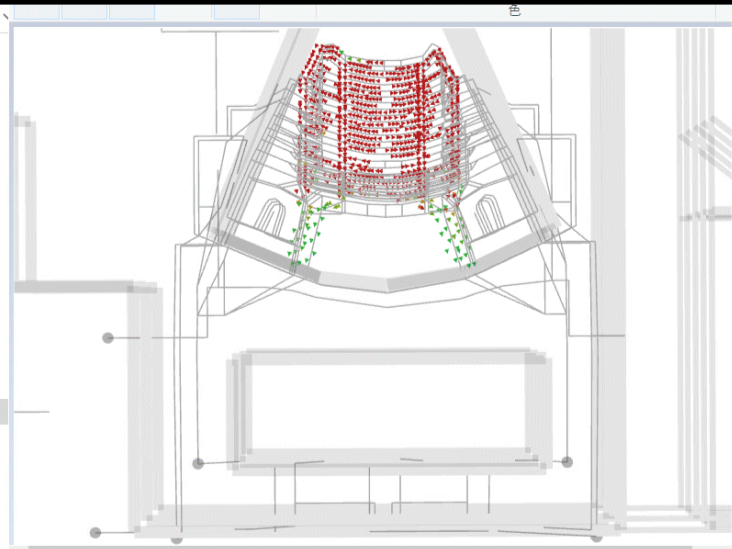
☒ Change agent color depending on speed

ここにチェック
(ただし遅くなる)

lect-crowdwalk > work > log > cap

名前

- capture000000.png
- capture000001.png
- capture000002.png
- capture000003.png
- capture000004.png
- capture000005.png
- capture000006.png
- capture000007.png
- capture000008.png
- capture000009.png
- capture000010.png
- capture000011.png
- capture000012.png
- capture000013.png
- capture000014.png
- capture000015.png
- capture000016.png
- capture000017.png
- capture000018.png
- capture000019.png



避難完了時間をグラフ化

lect-crowdwark¥work¥log¥evacuatedAgent.csv を利用

evacuatedAgent.csv - メモ帳

ファイル(F) 編集(E) 書式(O) 表示(V) ヘルプ(H)

EXIT_TEST|EXIT,EXIT_TEST2|EXIT,EXIT_TEST|EXIT ← 1行目ヘッダ

0,0,0
0,0,0
0,0,0
0,0,0
0,0,0
0,0,0
0,0,0

...

0,0,856
0,0,856
0,0,857
0,0,857
0,0,858
0,0,859
0,0,859
0,0,860
0,0,860
0,0,861
0,0,861
0,0,862
0,0,863
0,0,863
0,0,864
0,0,864
0,0,865
0,0,866
0,0,866
0,0,867
0,0,867
0,0,868

```
import pandas as pd
import matplotlib.pyplot as plt
```

CSVファイルを読み込む

```
df = pd.read_csv('work¥log¥evacuatedAgent.csv', header=None, skiprows=1)
```

3番目の列を取得

```
col_3 = df.iloc[:, 2]
```

```
plt.plot(col_3)
```

```
plt.grid(True)
```

```
plt.xlim(0, 600)
```

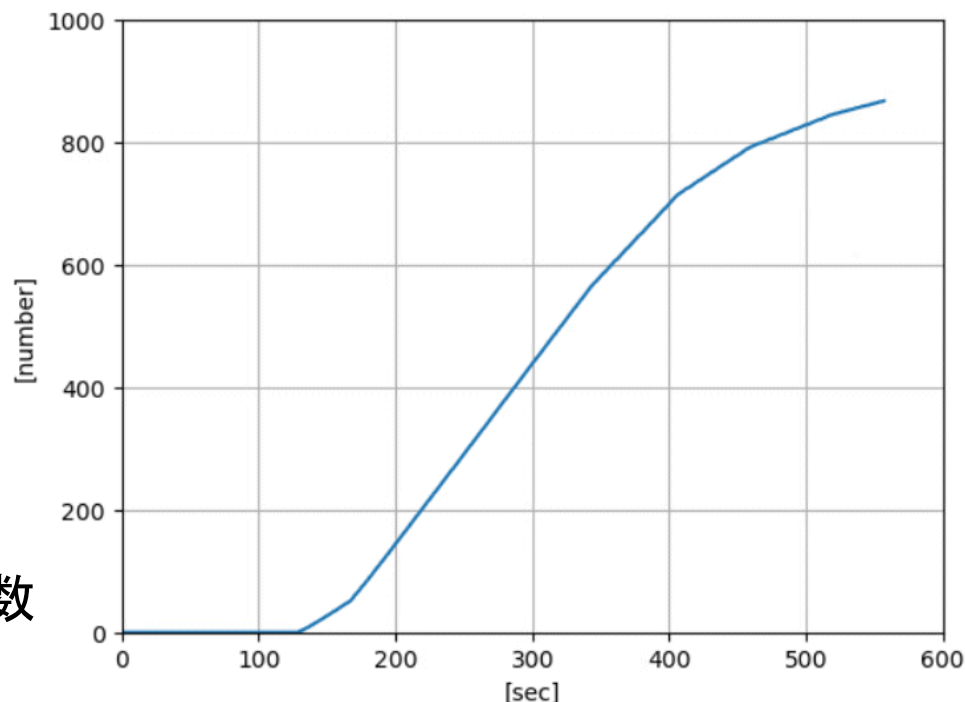
```
plt.ylim(0, 1000)
```

```
plt.xlabel('[sec]')
```

```
plt.ylabel('[number]')
```

```
plt.show()
```

← 3つ目の数字が避難完了人数
(1秒が1行)



レポート課題

本シミュレータを用いて安全に混乱なく速く避難できる誘導計画を立てて
A4 で 5ページ程度の避難誘導計画書を作ってください。
書式は自由です。最後の半ページくらいは講義の感想を書いて下さい。

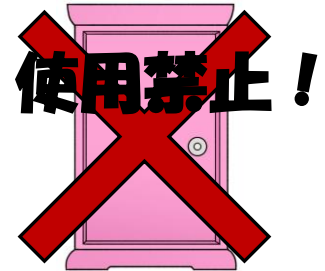
どういうシミュレーション結果をどう分析したのか、
その誘導計画の避難時間はどれくらいか、
なぜその誘導計画が良いと考えたのかをレポートに含めて下さい。
計画を考えるために作ったプログラムやグラフ、図表、
誘導の張り紙などを含めても構いません。

本レポートは積極的に生成 AI を利用することを推奨しますが、
利用した場合にはどの生成 AI にどんなプロンプトを与えたのかについて
試行の過程を含めて全て記述するとともにそれによって自分自身に
何が得られたかを別添資料として付けて下さい。

2週間後の授業開始までにmanabaで提出(※切厳守)

こんな方法が考えられるかも・・・

- ドア2と5には使用禁止という貼り紙を貼って鍵をかける(それでいいのか?)
- 各ブロック $6 \times 22 = 132$ を 8 つの扉のどこに誘導するかを全通りでシミュレーション
 $132^8 = 9$ 京2000兆通り試してみる(30億年くらい・・・)
- 左右対称性を利用して計算量を減らす(256分の1に)
- ドアに近い人をドアへ誘導し、それ以外の真ん中の人をどう誘導するかをシミュレーションで検証する



こんな方法が考えられるかも・・・

- 最も速い避難方法が最適な避難誘導とは限りません
少々誤った経路を選択した人がいても時間変化が
小さい誘導がいい誘導だと思います
- まずは最も良さそうな避難計画を手で作ってみて、
それを少しずつ変化させる形で最適化していく

来たれ筑波へ！

若き研究者たち



産総研インターンの募集

夏季：8月中旬から9月下旬で実働35日（7週間）
春季：2月中旬から3月下旬で実働35日（7週間）

※ 若干の期間の変更は応相談です
※ 旅費と宿泊費は負担します（日当は出ません）
※ 大西の受け入れ条件です

これまでの研究テーマ例

- ・ 歩行動作解析によるPDR利用者の同定
- ・ 動画画像処理による徘徊の可視化と検出
- ・ 超混雑環境における群集領域の抽出
- ・ 仮想情報と現実人流の相互伝播の解析
- ・ 人の流れの解析とシミュレーションの融合



こういう研究に興味を持った方は

インターン募集！

夏季インターンは **6/15** 応募〆切
実働30日以上

一例として 8/10～9/30(実働33日)

4年で大西研究室に配属
修士で連携大学院に進学を考える方を優先
(もちろん双方が合意すればの話)

インターン後は
学部3,4年は週2程度テクニカルスタッフとして
修士はリサーチアシスタントとして給料を支給

興味のある方は

<http://onishi-lab.jp/joinus/intern/index.html>

産総研 大西 インターン で検索して申し込みを

実証フィールドで計測

- ① 深層学習の最適化による群衆計測
- ② シミュレーションによる群衆誘導の最適化

劇場

新国立劇場 / 北九州芸術劇場
避難体験オペラコンサート

花火大会

門司港 / みなとみらい
関門会場花火大会



スタジアム

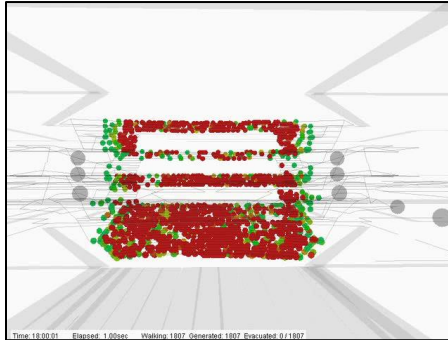
国立競技場 / カシマスタジアム / 東京ドーム /
日本代表戦、Jリーグ、プロ野球



実証フィールドで検証

劇場

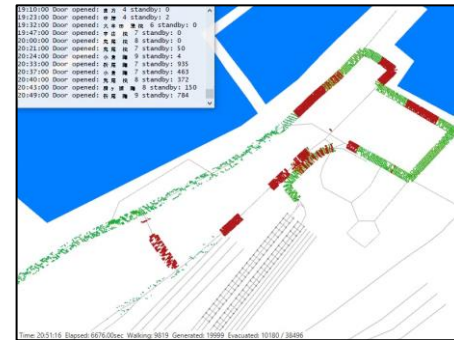
新国立劇場 / 北九州芸術劇場
避難体験オペラコンサート



- ① 深層学習の最適化による群衆計測
- ② シミュレーションによる群衆誘導の最適化

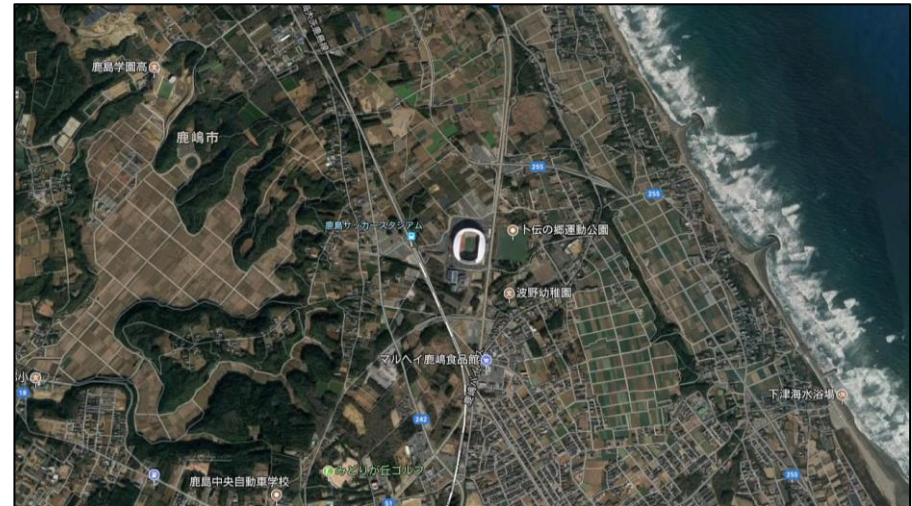
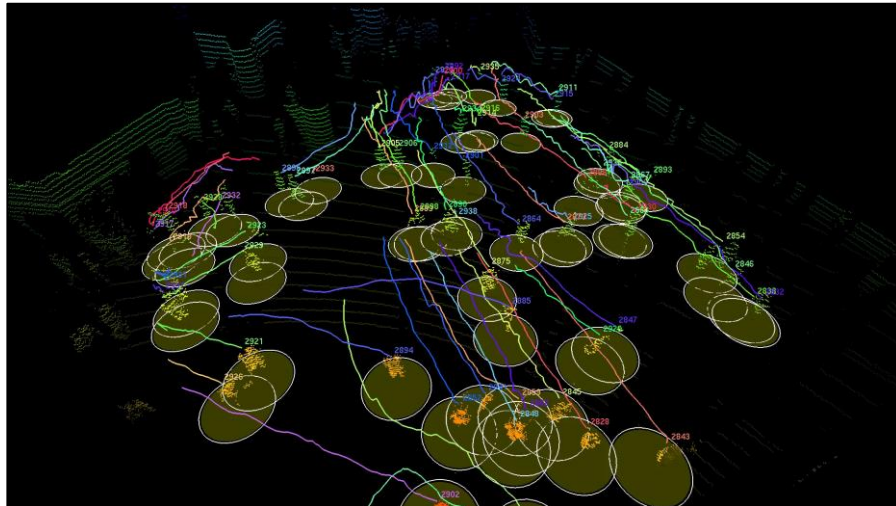
花火大会

門司港 / みなとみらい
関門会場花火大会



スタジアム

国立競技場 / カシマスタジアム / 東京ドーム /
日本代表戦、Jリーグ、プロ野球



来たれ筑波へ！

若き研究者たち



産総研インターンの募集

夏季：8月中旬から9月下旬で実働35日（7週間）
春季：2月中旬から3月下旬で実働35日（7週間）

※ 若干の期間の変更は応相談です
※ 旅費と宿泊費は負担します（日当は出ません）
※ 大西の受け入れ条件です

これまでの研究テーマ例

- ・ 歩行動作解析によるPDR利用者の同定
- ・ 動画画像処理による徘徊の可視化と検出
- ・ 超混雑環境における群集領域の抽出
- ・ 仮想情報と現実人流の相互伝播の解析
- ・ 人の流れの解析とシミュレーションの融合



こういう研究に興味を持った方は

インターン募集！

夏季インターンは **6/15** 応募〆切
実働30日以上

一例として 8/10～9/30(実働33日)

4年で大西研究室に配属
修士で連携大学院に進学を考える方を優先
(もちろん双方が合意すればの話)

インターン後は
学部3,4年は週2程度テクニカルスタッフとして
修士はリサーチアシスタントとして給料を支給

興味のある方は

<http://onishi-lab.jp/joinus/intern/index.html>

産総研 大西 インターン で検索して申し込みを